基于“递归嵌套式计算”的全低比特神经网络前向传播方法

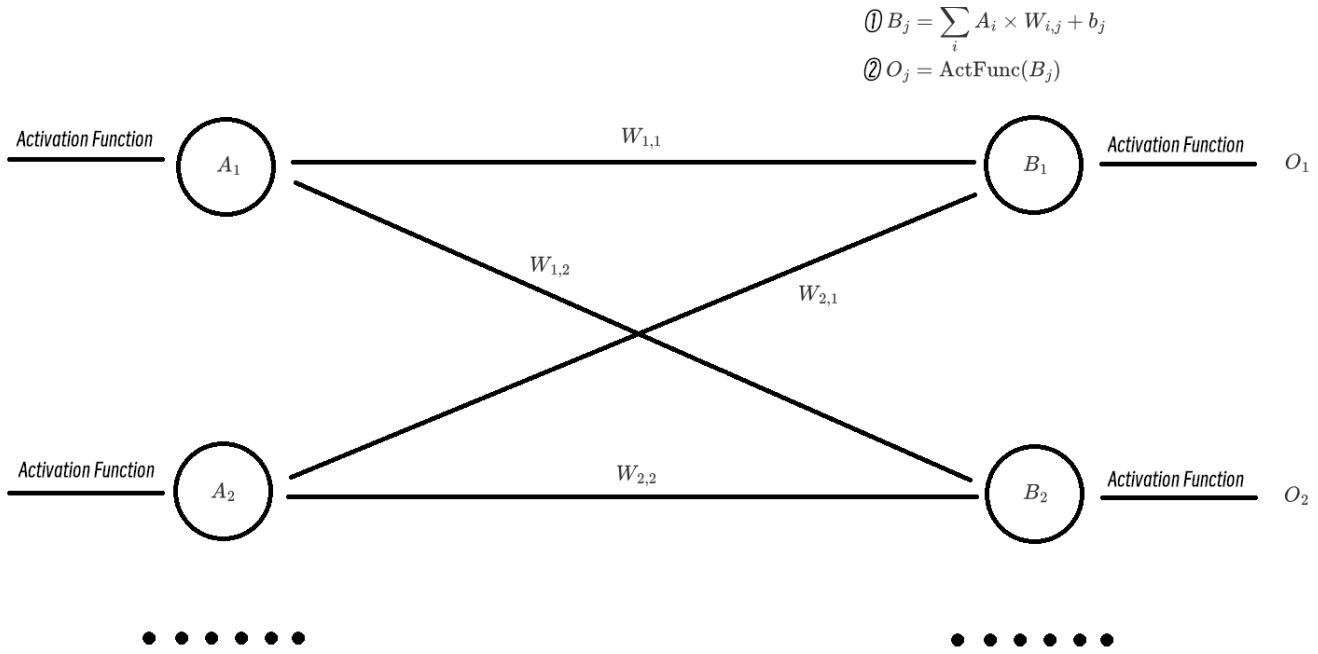
问题引入

在常见初始化和训练条件下，神经网络的权重和偏置通常在 $[-10, 10]$ 以内，很多激活函数的值域也很小，例如 $\mathrm{sigmoid}(x)$、$\tanh(x)$。可以发现，在常见设置下，**神经网络的参数往往集中在较小区间**，大部分数据类型如 C++ 标准 `float`、`double`，支持的值域达十的上百次方，神经网络前向传播时实际使用的数值范围仅占浮点能够表示的范围的一小部分，这为数值表示层面的优化提供了可能空间。此时会想到：能否利用神经网络参数值域集中的特点对其前向传播进行优化？可以，但不能运用于整个神经网络，因为在前向传播的线性求和阶段可能产生较大中间数值。

在线性变换阶段，由于多个神经元输出的加权求和，中间结果会随参与求和的神经元数量增加而增大，其值域可能显著扩大。**本文主要侧重于尽量不影响神经网络算法的前提下减少或避免对大值域中间结果的存储与计算。**

方法介绍

下图展示了神经网络前向传播的部分过程。



在该计算过程中：

- A_i 受限于激活函数，其值域较小；
- 权重 $W_{i,j}$ 与偏置 b_j 属于神经网络的可训练参数，在常见训练条件下通常也集中于较小区间；
- 因此乘积项 $A_i \times W_{i,j}$ 的数值范围同样较小。

然而，在对多个乘积项进行累加后，中间变量：

$$B_j = \sum_i A_i \times W_{i,j} + b_j$$

可能由于求和项数量较多而出现数值范围扩大的情况。

需要注意的是，B_j 最终仍将作为激活函数的输入，因此其输出值再次被压缩至较小区间。也就是说，虽然中间求和结果可能具有较大的数值范围，但这一中间值本身并不需要完整的大范围表示能力，只需保证最终激活输出的正确性即可。因此，该中间结果在数值表示层面存在优化空间。

为此，考虑构造一个二元函数 $\mathrm{f}(x, y)$，用于在不显式计算大值域中间结果的前提下，实现等价变换。

目标是构造函数 $\mathrm{f}(x, y)$，使其满足：

$$\mathrm{f}(\phi(x), y) = \phi(x + y)$$

（其中 $\phi(x)$ 表示值域较小的有界激活函数）

在此条件下，可以将原本的累加过程改写为递归嵌套形式：

$$\phi(B_j) = \ldots \mathrm{f}(\mathrm{f}(\phi(b_j), A_1 W_{1,j}), A_2 W_{2,j}) \ldots$$

通过这种递归构造方式，整个计算过程始终保持在激活函数输出的值域内，从而避免产生大范围的中间结果。

至此，问题转化为：如何构造满足上述等式条件的函数 $\mathrm{f}(x, y)$。然而，直接构造 $\mathrm{f}(x, y)$ 通常需要用到 $\phi^{-1}(x)$。该反函数的计算过程仍可能涉及大值域数值，从而重新引入数值范围扩大的问题。因此，需要进一步思考如何彻底规避这一现象。

结合优化的实际目标，可以注意到，实际工程中往往希望提高数值表示范围的利用效率，从而采用更小的数据类型以节省存储与计算资源。由于上述方法提高了数值表示范围的利用效率，因此可以考虑使用 8-bit、10-bit 或 12-bit 的数据类型。

对于 n-bit 数据类型，函数 $\mathrm{f}(x, y)$ 的输入组合数量为 $(2^n)^2 = 4^n$。当 $n = 8, 10, 12 \cdots$ 时，该规模是可接受使用查找表方式实现的。当然，本方法在具有优势的同时也具有一定的局限性，优势与局限性将在下文展开讨论。

实验与实现

为验证方法的可行性，本文以用于手写数字识别的前馈式全连接神经网络作为实现示例，训练数据来源于 MNIST 数据集。神经网络的代码实现与模型来源于作者个人项目 QNet，完整实现代码见 GitHub 仓库。模型已经提前使用 `float` 训练好，本方法仅用于优化前向传播，不用于模型训练。

一、神经网络结构

- 输入层：$28 \times 28 = 784$ 个神经元（单通道图像）；
- 隐藏层：两层，神经元数量分别为 256 与 64；
- 输出层：10 个神经元，对应数字 $0 \sim 9$；
- 激活函数：全层使用 $\mathrm{sigmoid}$；
- 每个非输入神经元均包含独立偏置项。

二、实现机制

1. 自定义小类型 `SamllType`

   为实现低比特数据类型，构造模板类：

   ```
   template <typename T, typename T1, int len, int V>
   class SamllType
   ```

   其核心特征：

- 总位宽：`len` 位（平衡了众多条件最终本实验使用 13-bit）；
- 最高位为符号位；
- 其余 `len-1` 位用来刻画绝对值；
- 表示区间为：$[-V, V]$。

采用线性映射方式（可根据不同情况做出相应调整）：

$$x \mapsto \mathrm{round}\left(\frac{|x|}{V} \cdot (2^{len-1} - 1)\right)$$

使用 `SamllType` 模板定义了两个类型：

类型	作用	范围
`T01`	激活值	$[-1, 1]$
`T05`	权重、偏置	$[-5, 5]$

2. 查表实现运算

出于方便，不仅 $\mathrm{f}(x, y)$ 使用查表实现，乘法也使用查表实现。

```
void preprocessing() {
    // nT 为 uint16_t
    for (nT i = 0; i < N; i++)
        for (nT j = 0; j < N; j++)
            __F[i][j] = T01(ff(T01(i).val(), T05(j).val())).get();
    for (nT i = 0; i < N; i++)
        for (nT j = 0; j < N; j++)
            __TIMES[i][j] = T05(T01(i).val() * T05(j).val()).get();
}
```

三、实验结果

1. 正确率与内存对比

数据类型	正确率	相对内存使用
`float`	95.7%	1
本方法（13-bit）	92.74%	$\frac{1}{2}$

说明：当前实现使用 `uint16_t` 存储 13-bit 数据，若采用压位存储，可将内存开销降至 $\frac{13}{32} \approx 40.6\%$，理论最大节省约 59.4% 内存。

需要指出的是，本实验仅统计模型参数存储开销，不包含查找表内存开销，表可看作长期存在的常量，可多程序共用，不计入单个程序的开销。

2. 正确率损失分析

绝对损失：

$$95.7\% - 92.74\% = 2.96\%$$

约 3% 正确率下降。

四、效果总结

实验结果表明，在 13-bit 数据类型下，该方法仅产生约 3% 的正确率损失，最多能降低约 59.4% 的内存使用。

这验证了：

> 在激活函数值域内进行递归嵌套式合并运算，可以基本替代传统累加法，并在可接受精度损失下实现全低比特神经网络前向传播。

优势与局限

一、优势

1. 降低累加器位宽需求

   在低 bit 的场景下，传统实现通常采用：

   - 低 bit 乘法；
   - 高 bit 累加器。

   以避免求和过程中的溢出问题。

   本方法降低对高精度累加器的依赖为，**全低比特实现提供可能。**

2. 若采用专用的硬件加速此算法，例如将表写入硬件，则能够在不影响性能的前提下减小内存消耗。

二、局限与解决方案

1. 激活函数限制

   - 激活函数 $\phi(x)$ 必须是有界函数且值域较小。
   - 激活函数 $\phi(x)$ 必须可逆。

   因此不适用于 ReLU。

2. 查找表规模指数增长

   对于 n-bit 数据类型，查找表规模为：

   $$4^n$$

   当 n 增大时存储需求指数增长，不适用于宽比特数据类型。

3. 嵌套式调用 $\mathrm{f}(x,y)$ 导致误差累积

   解决方案：传统的 n-bit 数据类型，2^n 个状态中实际可能用到的不足 10%，**本方法通过压缩值域提升状态利用率，从而提高精度。**一定程度上弥补了误差。

4. $\mathrm{f}(x,y)$ 没有结合律，无法并行计算

   解决方案：构造 $\mathrm{g}(\phi(x),\phi(y))=\phi(x+y)$，此时可以利用 $\mathrm{g}(x,y)$ 实现合并操作，可以分治并行计算。

5. 内存访问可能成为瓶颈

   解决方案：设计专门的硬件实现查表功能，但在通用 CPU、GPU 平台上该问题较难避免。